Verified Succession: A Recursive Context-Aware Multi-Agent Architecture for Software Engineering

Dylan Schwindt

Independent Research — February 2026

arXiv preprint *— Subject: Artificial Intelligence (cs.AI); Software Engineering (cs.SE); Multi-Agent Systems (cs.MA)*

Abstract

We introduce **Verified Succession**, a novel multi-agent orchestration architecture that addresses the fundamental constraint of context window exhaustion in large language model (LLM) agents performing complex software engineering tasks. Our architecture combines three primitives: (1) *context-triggered spawning*, where agents dynamically decompose their work into sub-swarms when approaching context limits; (2) *verified succession*, where terminating agents are replaced by fresh agents that independently audit prior work before continuing; and (3) *recursive dimensional scaling*, where this pattern applies at arbitrary depth, producing an emergent agent topology shaped by problem complexity rather than predetermined configuration.

Unlike existing orchestration patterns — sequential, concurrent, handoff, group chat, and magentic — which assume a fixed agent topology defined at design time, Verified Succession produces **emergent topology**: the shape of the swarm is determined at runtime by the shape of the problem. Central to the design is the *verify-then-continue* handoff protocol, in which a successor agent spawns a short-lived verification sub-agent to audit the predecessor's artifacts before continuing work. This eliminates context pollution, filters dead-end reasoning paths, and introduces an error-correction layer at every succession boundary.

We present the formal architecture, analyze its theoretical properties including context efficiency, error propagation characteristics, and depth-bounded resource consumption, provide a concrete cost model with empirical token economics, and discuss the tradeoffs, caveats, and applicability boundaries of the approach against current single-agent and static multi-agent alternatives. We further introduce the **Specification-Driven Routing Pipeline** — a human-in-the-loop pre-execution phase that converts ambiguous problem statements into validated specification sheets, determines the appropriate execution architecture through concurrent multi-agent assessment, and addresses the single weakest error category (requirements misunderstanding, $p_v \approx 0.05$) by moving requirements validation to the front of the pipeline where human judgment is most effective.

Keywords: multi-agent systems, agentic swarms, context window management, agent orchestration, verified succession, recursive spawning, specification-driven routing, software engineering automation

---

1. Introduction

1.1 The Context Ceiling Problem

Large language models have demonstrated remarkable capability in software engineering tasks including code generation, debugging, refactoring, and architectural reasoning [1, 2]. However, all transformer-based LLMs operate within a fixed context window — a hard upper bound on the total tokens (input + output + reasoning) an agent can process in a single session. As of early 2026, context windows range from 128K to 1M tokens across frontier models [3], yet complex software engineering tasks routinely exceed these limits.

The consequences of context exhaustion are well-documented. Anthropic's BrowseComp evaluation found that **token usage explained 80% of performance variance** in agent tasks [4]. As context fills, models exhibit degraded attention over earlier tokens, reduced reasoning coherence, and increased hallucination rates [5]. Current mitigation strategies fall into three categories:

1. **Truncation and summarization** — lossy compression of prior context, sacrificing detail and nuance.
2. **Retrieval-augmented generation (RAG)** — swapping context segments in and out of an external store, introducing retrieval latency and relevance errors.
3. **Hard failure** — the agent simply cannot complete the task.

None of these approaches treat context exhaustion as a *structural signal*. We argue that hitting a context threshold is not a failure condition but a **spawning signal** — a natural trigger for task decomposition and delegation.

1.2 The Limitations of Static Multi-Agent Orchestration

The emergence of multi-agent architectures [6, 7, 8] has partially addressed the context ceiling by distributing work across agents with independent context windows. Microsoft's Azure Architecture Center identifies five canonical orchestration patterns: sequential, concurrent, group chat, handoff, and magentic [9]. Organizations using multi-agent architectures report 45% faster problem resolution and 60% more accurate outcomes compared to single-agent systems [10].

However, all five patterns share a critical assumption: **the agent topology is defined at design time**. The architect decides upfront how many agents exist, what each specializes in, and how they communicate. This creates a mismatch when problem complexity is unknown in advance — a defining characteristic of software engineering tasks. A seemingly simple bug fix may require deep investigation across multiple subsystems; a straightforward feature may touch unexpected dependency chains.

1.3 Contributions

This paper introduces the **Verified Succession** architecture, which makes five primary contributions:

1. **Context-triggered recursive spawning.** Agents monitor their own context utilization and, upon crossing a configurable threshold, decompose remaining work into a sub-swarm. This applies recursively at arbitrary depth, producing fractal agent topologies that scale with problem complexity.
2. **The verify-then-continue handoff protocol.** When an agent within a sub-swarm exhausts its context, it terminates and is replaced by a fresh agent. Critically, the successor does not inherit the predecessor's context. Instead, it spawns a short-lived verification sub-agent that independently audits the predecessor's artifacts and produces a lean, verified progress report. The successor begins with a near-empty context window containing only the original task and the verified report.
3. **Emergent topology.** Unlike all canonical orchestration patterns, the shape of the agent swarm is not predetermined. It emerges at runtime based on actual problem complexity, context consumption rates, and task decomposition decisions made by the agents themselves.
4. **Concrete cost quantification and applicability boundaries.** We provide a detailed cost model grounded in current API pricing and empirical token economics, an honest analysis of what the architecture sacrifices, and a decision framework identifying when the approach is and is not justified.
5. **Specification-Driven Routing Pipeline.** A production-grade pre-execution phase that converts raw problem statements into validated specifications through human-in-the-loop interviewing, assesses complexity via concurrent multi-agent evaluation, and routes to the appropriate execution architecture. This addresses the architecture's weakest error category — requirements misunderstanding — by elevating its detection probability from $p_v \approx 0.05$ to $p_v \approx 0.90+$.

---

2. Related Work

2.1 Multi-Agent Orchestration Patterns

The systematization of multi-agent orchestration patterns has accelerated in 2025-2026. Microsoft's Azure Architecture Center [9] defines five patterns along two axes: coordination style (linear, parallel, conversational, delegative, adaptive) and routing determinism (static vs. dynamic). Google's ADK [11] and Salesforce's Enterprise Agentic Architecture [12] offer similar taxonomies. All assume design-time topology definition.

The **Magentic** pattern [9] comes closest to our work in its adaptive planning capability — a manager agent dynamically builds and refines a task ledger. However, Magentic does not address context exhaustion, does not support recursive spawning, and does not include verified handoff between agent generations.

2.2 Dynamic Agent Spawning

AgentSpawn [13] (February 2026) introduces dynamic agent collaboration triggered by runtime complexity metrics, with automatic memory transfer during spawning events. Their results demonstrate 34% higher task completion rates over static baselines and 42% reduction in memory overhead through selective context slicing. AgentSpawn addresses five gaps: memory continuity, skill inheritance, task resumption, runtime spawning, and concurrent coherence. Our work extends this by introducing *verified succession* as the handoff mechanism, replacing direct memory transfer with independent audit.

Kimi Agent Swarm [14] demonstrates runtime swarm composition at scale (100+ sub-agents), where a CEO agent dynamically recruits specialist agents. **MetaSwarm** [15] implements recursive orchestration where Swarm Coordinators spawn Issue Orchestrators, which spawn sub-orchestrators — a "swarm of swarms" pattern. Neither system addresses the context exhaustion trigger or verified handoff.

2.3 Context Window Management

The context ceiling problem has received significant attention. The pointer-based abstraction approach [16] enables agents to reference stored outputs via memory pointers rather than carrying raw data, achieving approximately 7x token reduction. LangChain's context engineering guidelines [17] recommend structured approaches to managing what context enters each agent's window. These approaches are complementary to our architecture and can serve as implementation mechanisms for the state store component.

2.4 Verification in Multi-Agent Systems

VERIMAP [18] (Megagon AI) introduces verification-aware planning for multi-agent LLM systems, embedding verification functions at every subtask boundary. Their core finding — that *"failures increasingly arise from coordination rather than raw reasoning ability"* — directly motivates our verified succession protocol. VERIMAP operates at the planning level with persistent agents; our contribution applies verification at the agent *succession* boundary, where one agent terminates and another continues its work.

Multi-Agent Code Verification via Information Theory [19] provides formal grounding for measuring verification quality across agent boundaries, applicable to our verifier sub-agent design.

2.5 The Partial Completion Problem

Karapetyan [20] identifies the "partial completion" problem in LLM agents — tasks that are started but not finished due to context exhaustion, attention degradation, or reasoning failures. Current solutions include episodic memory (writing summaries when deleting step entries to keep context clean) and checkpoint-based resumption. Our architecture addresses partial completion structurally: the verified succession protocol ensures that partially completed work is audited, corrected if necessary, and continued by a fresh agent with full context capacity.

2.6 Multi-Agent Token Economics

Recent empirical work has quantified the token overhead of multi-agent systems. Kim et al. [23] find that multi-agent systems consume 4-220x more input (prefill) tokens than single-agent systems, with inter-agent communication, role definitions, and system prompt repetition as primary overhead sources. Even with optimizations, multi-agent systems require 2-12x more tokens for response generation. Complementary work [24] analyzes where tokens are spent in agentic coding tasks, finding that unconstrained agents can cost $5-8 per software engineering issue.

2.7 Scaling Laws for Agent Systems

Google Research [25] evaluated 180 agent configurations and found that task structure — not agent count — determines whether multi-agent architectures succeed. For parallelizable tasks, centralized coordination improved performance by 80.9% over a single agent. For sequential tasks, all multi-agent variants *degraded* performance by 39-70%, as communication overhead "fragmented the reasoning process, leaving insufficient cognitive budget for the actual task." Critically, independent agents amplified errors by 17.2x without mutual validation mechanisms, while centralized systems contained amplification to 4.4x. These findings directly motivate our architecture's verified handoff: succession boundaries serve as the validation mechanism that prevents error amplification.

---

3. Architecture

3.1 System Overview

The Verified Succession architecture operates across three organizational levels:

```
Level 0: ORCHESTRATOR
         │
         ├── Concurrent dispatch of top-level agents
         │
Level 1: Agent_A ── Agent_B ── Agent_C ── ... ── Agent_N
         │
         ├── Context-triggered decomposition
         │
Level 2: Sub-Orchestrator_A
         ├── Agent_A.1
         ├── Agent_A.2 → [succession] → Agent_A.2' → [succession] → Agent_A.2''
         └── Agent_A.3
              │
Level 3+:    (recursive, depth-bounded)
```

Definition 1 (Agent). An agent α is a tuple (T, C, θ, S) where:
- T is the task description (immutable)
- C is the current context window contents (mutable, bounded by C_{max})
- θ is the context threshold (configurable, default 0.6 × C_max)
- S is a reference to the shared state store

Definition 2 (Swarm). A swarm Σ is a set of agents $\{\alpha_1, \ldots, \alpha_k\}$ managed by a sub-orchestrator ω, operating on decomposed subtasks of a parent agent's original task.

Definition 3 (Succession Event). A succession event occurs when $|C(\alpha)| \geq \theta(\alpha)$ for some agent α within a swarm. The agent writes its artifacts to the state store, terminates, and the sub-orchestrator spawns a successor.

3.2 Component Architecture

The system comprises five components:

3.2.1 The Orchestrator

The top-level orchestrator implements the concurrent (fan-out/fan-in) pattern [9]. It receives the overall task, decomposes it into independent subtasks, and dispatches them to top-level agents in parallel. It aggregates results when all top-level agents complete. The orchestrator maintains a task manifest tracking the status of each top-level agent.

3.2.2 Agents

Each agent operates with an independent context window. Agents are instrumented with a **context monitor** that tracks current utilization as a fraction of C_{max}. When utilization crosses threshold θ, the agent enters the **decomposition phase** (Section 3.3) rather than continuing to degrade.

3.2.3 Sub-Orchestrators

When an agent decomposes, it spawns a sub-orchestrator to manage the resulting sub-swarm. The sub-orchestrator is a lightweight coordinator responsible for:
- Managing the lifecycle of child agents
- Handling succession events (spawning replacements, dispatching verifiers)
- Aggregating results when all subtasks complete
- Enforcing depth and resource limits

The sub-orchestrator is distinct from the decomposing agent itself. Agent A does not manage its children directly — it delegates to Sub-Orchestrator A' and waits for the aggregated result. This separation prevents the parent agent's context from being consumed by coordination overhead.

3.2.4 Verification Sub-Agents

Verification sub-agents (hereafter *verifiers*) are short-lived, single-purpose agents spawned during succession events. A verifier:
1. Reads the terminated agent's artifacts from the state store
2. Evaluates completeness, correctness, and remaining work
3. Produces a structured verification report
4. Returns the report to the successor agent
5. Terminates (its context is discarded)

Verifiers are bounded: their report must not exceed `0.05 × C_max` of the successor's context window, ensuring the successor retains maximum headroom.

3.2.5 State Store

An external, persistent key-value store accessible to all agents. Agents write **artifacts** (not context) to the store upon termination. The state store holds:
- Task manifests (structured descriptions of subtask decompositions)
- Work artifacts (code, analysis results, structured outputs)
- Completion status (per-subtask progress tracking)
- Succession metadata (lineage of agent generations per subtask)

The state store is the system's durable memory. Individual agents are ephemeral; the store persists across all succession events.

3.3 The Decomposition Phase

When agent `α` crosses threshold `θ`:

```
DECOMPOSITION PROTOCOL
──────────────────────
1. α analyzes remaining work in current context
2. α identifies k independent (or minimally dependent) subtasks
3. α writes a task manifest to the state store:
   {
     parent: α.id,
     subtasks: [
       { id: "A.1", description: "...", dependencies: [] },
       { id: "A.2", description: "...", dependencies: [] },
       { id: "A.3", description: "...", dependencies: ["A.1"] }
     ],
     context_at_decomposition: |C(α)| / C_max,
     artifacts_so_far: [pointers to any work α completed before threshold]
   }
4. α spawns Sub-Orchestrator ω with the task manifest
5. α enters WAIT state (or terminates if no further coordination needed)
6. ω spawns agents α.1, α.2, ..., α.k with individual subtask descriptions
```

Critical design constraint: The decomposition must occur *before* the agent's reasoning quality degrades. This is why the threshold θ must leave sufficient headroom (Section 4.2) for the meta-reasoning required to decompose well.

3.4 The Verified Succession Protocol

When agent $\alpha.j$ within a sub-swarm crosses threshold θ:

```
VERIFIED SUCCESSION PROTOCOL
____________________________

Phase 1: Termination
  1. α.j writes work artifacts to state store
  2. α.j writes a self-assessment manifest:
     {
       subtask: "original task description",
       artifacts: [pointers],
       self_assessment: "completed X, Y; started Z",
       blockers_discovered: [...],
       generation: g
     }
  3. α.j terminates

Phase 2: Successor Spawning
  4. Sub-Orchestrator ω spawns successor α.j' with ONLY:
     - Original subtask description T(α.j)
     - Pointer to α.j's artifacts in state store
     - Generation counter: g + 1

Phase 3: Verification
  5. α.j' spawns Verifier sub-agent V with:
     - The subtask description
     - Access to α.j's artifacts and self-assessment in state store
  6. V reads and evaluates artifacts independently:
     - Checks each claimed completion against actual artifacts
     - Identifies errors, omissions, or drift from the subtask
     - Assesses remaining work
  7. V produces a structured Verification Report:
     {
       verified_complete: ["list of confirmed-done items"],
       issues_found: ["item X: expected 4 roles, only 1 implemented"],
       remaining_work: ["list of outstanding items"],
       recommended_approach: "brief strategic guidance",
       confidence: 0.0-1.0
     }
  8. V returns report to α.j'
  9. V terminates

Phase 4: Continuation
  10. α.j' incorporates the verification report (~5% of C_max)
  11. α.j' continues work from the verified state
      Context utilization at start: ~8% (task + report)
      Available headroom: ~92%
```

3.5 Error Recovery During Succession

The verification phase introduces a natural error-correction mechanism. When the verifier detects issues, the sub-orchestrator ω selects from three recovery strategies:

Strategy 1: Corrective Continuation. If issues are minor (e.g., incomplete implementation of a known requirement), the verification report includes the issues, and `α.j'` addresses them as part of its continued work. This is the default path.

Strategy 2: Clean Restart. If the verifier's confidence score falls below a threshold `φ` (default 0.3), indicating that the predecessor's work is fundamentally flawed, `ω` spawns `α.j'` with only the original task description and no reference to prior artifacts. The predecessor's work is abandoned.

Strategy 3: Redecomposition. If the verifier determines the subtask itself is too complex (indicated by repeated succession events with low progress), `ω` escalates to the parent agent or orchestrator, recommending further decomposition. The subtask is split into smaller units.

Strategy 4: Escalation. If recovery is not possible at this level (e.g., the subtask depends on external information that no agent possesses), `ω` reports the blocker to the parent orchestrator with the verifier's assessment.

3.6 Result Aggregation

When all agents in a sub-swarm complete:

```
AGGREGATION PROTOCOL
────────────────────
1. Sub-Orchestrator ω collects final artifacts from all child agents
2. ω performs completeness check against original task manifest
3. ω resolves contradictions between sub-agents (if any)
4. ω produces aggregated result for parent agent
5. Parent agent (or top-level orchestrator) receives result
6. ω terminates, sub-swarm is dissolved
```

For the top-level orchestrator, aggregation follows the standard concurrent fan-in pattern: independent results from Agent_A, Agent_B, …, Agent_N are combined into the final output.

---

4. Theoretical Analysis

4.1 Context Efficiency

Theorem 1 (Context Utilization Bound). *In a Verified Succession architecture with threshold θ and verifier report bound β, the maximum context utilization of any successor agent at the point of task resumption is:*

```
U_start(α') = |T| / C_max + β
```

where $|T|$ is the token count of the original subtask description and $\beta \leq 0.05$.

Proof sketch. The successor `α'` receives only the original task description `T` and the verifier's report (bounded by `β × C_max`). No other context is transferred. Therefore, the initial utilization is `(|T| + β × C_max) / C_max = |T|/C_max + β`. For typical subtask descriptions, `|T|/C_max ≪ 0.05`, yielding `U_start ≈ 0.05-0.10`. ∎

Corollary 1. *The effective context available per unit of work is maximized under Verified Succession compared to context-dump handoff, where U_start approaches θ (the termination threshold).*

Compare the three handoff strategies:

Strategy	U_start of successor	Effective headroom
Context dump	~0.40-0.55	~0.45-0.60
Compressed summary	~0.15-0.25	~0.75-0.85
Verified Succession	~0.05-0.10	**~0.90-0.95**

4.2 Threshold Selection

Proposition 1 (Decomposition Headroom). *The context threshold θ must satisfy:*

```
θ ≤ 1 - D(k)
```

where D(k) is the context cost of decomposing into k subtasks, including analysis, manifest writing, and sub-orchestrator initialization.

Empirically, decomposition into 2-4 subtasks requires approximately 0.15-0.25 of `C_max` for adequate meta-reasoning (analyzing remaining work, identifying subtask boundaries, writing structured manifests). This yields:

```
θ_optimal ∈ [0.55, 0.70]
```

The default value of `θ = 0.60` provides a 40% headroom budget: approximately 15-25% for decomposition meta-reasoning and 15-25% safety margin for tool responses and reasoning during decomposition.

Proposition 2 (Adaptive Threshold). *A context-type-aware threshold outperforms a fixed threshold. Define:*

```
θ_adaptive(α) = θ_base - λ × tool_density(α)
```

where tool_density(α) is the ratio of tool-call tokens to total tokens in α's context, and λ is a sensitivity parameter. Agents with high tool density (many API calls, large responses) should spawn earlier because their context is more "fragile" — tool outputs are less compressible and less useful as reasoning progresses.

4.3 Depth Bound Analysis

Theorem 2 (Depth Bound). *For a task requiring T_total tokens of effective reasoning and agents with context capacity C_max and threshold θ, the maximum recursion depth d is:*

```
d = ⌈log_k(T_total / (θ × C_max))⌉
```

where k is the average branching factor (number of subtasks per decomposition).

Example. Consider a task requiring 2M tokens of effective reasoning, with `C_max = 200K`, `θ = 0.6`, and `k = 3`:

```
d = ⌈log_3(2,000,000 / 120,000)⌉ = ⌈log_3(16.67)⌉ = ⌈2.56⌉ = 3
```

Three levels of recursion suffice, with at most `3^3 = 27` leaf agents.

Theorem 3 (Resource Bound). *The maximum number of agents spawned (including verifiers) for a task of depth d with branching factor k and maximum s succession events per subtask is:*

```
N_max = Σ_{i=0}^{d} k^i × (1 + s_i × 2)
```

The factor of 2 accounts for each succession event producing both a successor and a verifier. The +1 accounts for the initial agent.

With `d = 3`, `k = 3`, and at most `s = 2` successions per subtask:

```
Level 0: 1 orchestrator
Level 1: 3 agents × (1 + 2×2) = 15
Level 2: 9 agents × (1 + 2×2) = 45
Level 3: 27 agents × (1 + 2×2) = 135
Total: ~196 agents (including verifiers)
```

This provides a **predictable upper bound** on resource consumption, critical for cost management in production deployments.

4.4 Error Propagation

Theorem 4 (Error Attenuation). *Let p_e be the probability of an agent producing an error in its output, and p_v be the probability of a verifier detecting an existing error. The probability of an error surviving a succession boundary is:*

```
P(error survives) = p_e × (1 - p_v)
```

For k succession events in sequence, the probability of an error from the first agent reaching the final output is:

```
P(error persists through k successions) = p_e × (1 - p_v)^k
```

With realistic values of `p_e = 0.15` (15% error rate) and `p_v = 0.80` (80% verifier detection rate):

```
After 1 succession: 0.15 × 0.20 = 0.030 (3.0%)
After 2 successions: 0.15 × 0.04 = 0.006 (0.6%)
After 3 successions: 0.15 × 0.008 = 0.001 (0.1%)
```

Each succession boundary acts as an error filter. This is a fundamental advantage over context-dump approaches, where errors transfer directly with probability `p_e` at each stage. The architecture exhibits exponential error attenuation with depth — the more succession events occur, the more reliable the final output becomes.

Corollary 2 (Contrast with Context Dump). *In a context-dump handoff, errors are transferred with probability ≈ 1 (they exist in the dumped context and are treated as ground truth by the successor). Verified Succession reduces error propagation by a factor of:*

```
Improvement = 1 / (1 - p_v)^k
```

For k = 2 and p_v = 0.80, this is a 25× improvement in error propagation rate.

4.5 The Dead-End Filtering Property

Definition 4 (Dead-End Tokens). Dead-end tokens are context consumed by reasoning paths, tool calls, and hypotheses that were explored and ultimately abandoned or found irrelevant.

In conventional agent operation, dead-end tokens accumulate in the context window and cannot be selectively removed. They consume attention capacity and may mislead subsequent reasoning. We estimate that for complex software engineering tasks, dead-end tokens constitute 20-40% of total context consumption.

Proposition 3. *Verified Succession eliminates 100% of dead-end tokens at each succession boundary, since the successor receives only the verified report (which contains no dead-end content) and the original task description.*

This is a unique property not shared by any context management technique that operates *within* a single agent's session (summarization, RAG, sliding windows). These techniques can reduce but never fully eliminate dead-end token influence. Verified Succession achieves complete elimination by starting fresh.

---

5. Quantification, Cost Analysis, and Tradeoffs

A novel architecture is only as valuable as its honest accounting. This section provides concrete cost models grounded in current API pricing, identifies what the architecture sacrifices, quantifies what it gains, and defines the boundaries of applicability.

5.1 Token Overhead Reality

Multi-agent systems carry inherent overhead. Kim et al. [23] measure that multi-agent systems consume **4-220x more input tokens** than single-agent alternatives, with three primary overhead sources:

1. **System prompt repetition.** Each agent receives its own system prompt, role definition, and tool descriptions. For N agents, this is approximately N × |system_prompt| tokens of pure overhead.
2. **Inter-agent communication.** Messages between agents are natural language processed by LLM calls. Each message exchange becomes an additional inference call.
3. **Coordination reasoning.** Orchestrators and sub-orchestrators consume tokens deciding what to delegate, how to aggregate, and when to intervene.

Verified Succession mitigates sources (1) and (2) through its design: successors receive only the original task plus a lean verifier report (not the predecessor's full message history), and verifiers are short-lived (their prompt overhead amortizes over a single focused task). However, source (3) remains — sub-orchestrators and the decomposition protocol consume tokens for coordination.

Measured overhead range for Verified Succession: We estimate 4-10x input token consumption relative to a single-agent baseline for typical tasks (Section 5.2), placing the architecture at the lower end of the multi-agent overhead spectrum.

5.2 Cost Model

We construct a concrete cost model using Claude Opus 4.6 pricing as of February 2026 [26]:

- Input tokens: $5.00 per million tokens
- Output tokens: $25.00 per million tokens
- Prompt cache reads: $0.50 per million tokens (90% reduction)

5.2.1 Scenario A: Single Agent Baseline

A single agent attempts a complex software engineering task (e.g., implementing an RBAC authentication system). The task exceeds the agent's effective capacity; quality degrades past 60% context utilization.

```
1 agent session, quality degrades past context threshold

Input tokens:  ~150K (prompt + accumulated tool responses)
Output tokens: ~40K  (reasoning + code generation)

Cost: (150K × $5/1M) + (40K × $25/1M)
    = $0.75 + $1.00
    = $1.75

Estimated task completion: ~75%
Error rate in final 30% of output: 15-25%
```

5.2.2 Scenario B: Verified Succession — Typical Case

The task decomposes into 3 concurrent top-level agents. One agent (B) spawns a sub-swarm of 3. One sub-agent (B.2) undergoes a single succession event.

```
Agent inventory:
  3 top-level agents                          = 3 sessions
  1 sub-orchestrator (lightweight)            = 1 session
  3 sub-agents for Agent B                    = 3 sessions
  1 successor (B.2')                          = 1 session
  1 verifier (short-lived)                    = 1 session
                                        Total = 9 sessions

Token consumption per category:
  Top-level agents:   3 × (80K in + 25K out)  = 240K in,  75K out
  Sub-orchestrator:   1 × (30K in + 10K out)  =  30K in,  10K out
  Sub-agents:         3 × (100K in + 25K out) = 300K in,  75K out
  Successor (B.2'):   1 × (90K in + 25K out)  =  90K in,  25K out
  Verifier:           1 × (55K in + 5K out)   =  55K in,   5K out
                                       Total: 715K in, 190K out

Gross cost: (715K × $5/1M) + (190K × $25/1M)
          = $3.58 + $4.75
          = $8.33

With prompt caching (~30% cache hit rate on shared prompts):
  Adjusted input: 500K full-price + 215K cached
  = (500K × $5/1M) + (215K × $0.50/1M) + (190K × $25/1M)
  = $2.50 + $0.11 + $4.75
  = $7.36

Estimated task completion: ~95%
Error rate: <3% (verified outputs)
```

5.2.3 Scenario C: Verified Succession — Deep Recursion

Depth 3, branching factor 3, up to 2 successions per subtask. Represents a highly complex task (e.g., full-system refactoring across multiple subsystems).

```
Agent inventory: up to ~196 sessions (Theorem 3)
Practical estimate: ~50 active sessions (not all subtasks
  require full depth or successions)

Estimated tokens: ~5M input, ~1.5M output

Gross cost: (5M × $5/1M) + (1.5M × $25/1M)
          = $25 + $37.50
          = $62.50

With caching + Haiku-tier verifiers ($1/$5 per million):
  ≈ $40-50
```

5.2.4 Scenario D: Worst-Case Upper Bound

All 196 agent sessions fully utilized, no caching, all sessions use Opus-tier models.

```
Estimated tokens: ~15M input, ~4M output

Cost: (15M × $5/1M) + (4M × $25/1M)
    = $75 + $100
    = $175
```

5.2.5 Cost Comparison Summary

Scenario	Agent Sessions	Cost	Task Completion	Error Rate
Single agent (degrades)	1	$1.75	~75%	15-25%
Single agent + RAG/summary	1	$2.50	~85%	10-15%
VS — Typical	**9**	**$7-8**	**~95%**	**<3%**
VS — Deep	**~50**	**$40-60**	**~97%**	**<1%**
VS — Worst case	~196	$80-175	~98%+	<0.5%

The cost multiplier ranges from 4-5x for typical tasks to 20-100x for deep recursion. This premium must be justified by the value of the task.

5.3 Latency Profile

Every succession event introduces sequential overhead that cannot be parallelized:

Phase	Duration	Notes
Agent writes artifacts to store	5-10s	I/O bound, artifact size dependent
Agent terminates	~1s	Process cleanup
Successor spawns	2-5s	Model initialization, prompt loading
Verifier spawns + reads artifacts	10-20s	Largest component; artifact assessment
Verifier produces report	5-10s	LLM inference for structured report
Verifier terminates	~1s	Process cleanup
Successor incorporates report	2-5s	Prompt processing
Total per succession event	**~30-60s**	

Aggregated latency for representative scenarios:

Scenario	Succession Events	Coordination Overhead	Total Wall Clock
Single agent	0	0	3-5 min
VS — Typical (concurrent top-level)	1	~45s	8-15 min
VS — Deep (3 levels)	5-8 across tree	~4-8 min	20-40 min
VS — Worst case	15+ across tree	~10-15 min	45-90 min

For reference, a single LLM inference call takes approximately 800ms [27]. Multi-agent orchestration systems typically require 10-30 seconds per coordination round [27]. Prompt caching can reduce individual call latency by approximately 75% [27].

The latency tradeoff is directional: Verified Succession is 2-8x slower than a single agent in wall-clock time, but completes tasks that a single agent cannot finish at all or finishes with degraded quality.

5.4 What the Architecture Sacrifices

5.4.1 Cost Efficiency on Simple Tasks

Verified Succession has a **minimum overhead floor**. The context monitoring, threshold checking, and decomposition infrastructure run regardless of task complexity. For tasks that fit comfortably within a single agent's context window (< 50% utilization), this machinery adds cost without benefit. The architecture should not activate for simple tasks; the context monitor should confirm that the threshold is never crossed and the agent completes normally.

5.4.2 Determinism and Reproducibility

The same task, run twice, will likely produce different agent topologies. An agent may cross the 60% threshold at different points depending on tool call ordering, LLM sampling, and environmental factors. Consequences:

- **Variable cost per run.** A task might cost $7 on one run and $12 on another.
- **Variable quality.** Different decompositions produce different subtask boundaries and potentially different solutions.
- **Debugging difficulty.** Reproducing a specific failure requires recreating the exact agent topology, which is stochastic.
- **Testing challenges.** Integration tests cannot assert on exact agent counts or topology shapes.

Mitigation: Log all decomposition decisions, succession events, and verifier reports to the state store lineage. Use this audit trail for post-hoc analysis rather than relying on reproducibility.

5.4.3 Decomposition Quality Risk

The entire architecture depends on a critical moment: when an agent at 60% context utilization decides how to decompose its remaining work. This agent must perform high-quality meta-reasoning — analyzing what's left, identifying subtask boundaries, assessing dependencies — with only 40% of its context budget remaining for this meta-task. A poor decomposition can:

- Create subtasks that are secretly interdependent (agents block each other or duplicate work)
- Miss a critical subtask entirely (gap in coverage discovered only at aggregation)
- Split along wrong boundaries (each sub-agent needs context that lives in a sibling's scope)

Google's finding [25] that communication overhead "fragmented the reasoning process" is directly applicable here: a bad decomposition fragments the problem in ways that make the fragments harder to solve than the whole.

5.4.4 State Store as Single Point of Failure

All agents read and write artifacts to a shared state store. Failure modes include:

- **Store unavailability:** Partially written artifacts corrupt the succession chain.
- **Concurrent write conflicts:** Two sibling agents writing to overlapping paths produce inconsistent state.
- **Storage growth:** Deep recursion with multiple generations can produce significant artifact accumulation.

The sub-orchestrator must enforce path isolation between concurrent siblings and implement transactional writes with rollback capability.

5.5 What the Architecture Gains

5.5.1 Breaking the Context Ceiling

This is the fundamental value proposition. Without Verified Succession, tasks exceeding a single context window either fail outright or complete with severely degraded quality. With it, there is no theoretical upper bound on task complexity.

Quantified capacity gain: A single Claude Opus 4.6 session provides ~200K tokens of effective reasoning. Verified Succession with depth 3 and branching factor 3 provides up to 200K × 27 leaf agents = **5.4 million tokens** of effective reasoning capacity, with error correction at every succession boundary.

This is not an incremental improvement. It is a **capability unlock** — enabling a class of tasks that was previously impossible for autonomous agents.

5.5.2 Error Attenuation (Stratified by Error Category)

The error attenuation theorem (Theorem 4) assumes a uniform detection probability `p_v`. In practice, `p_v` varies dramatically by error category. The following stratification provides a more accurate assessment:

Error Category	Detection Method	p_v Estimate	After 2 Successions
Type errors, missing imports	`tsc`, `cargo check`	~0.99	0.0001%
Test-catchable logic errors	Test execution	~0.85	0.34%
Subtle logic errors	LLM judgment	~0.40	5.4%
Architectural mistakes	LLM judgment	~0.15	10.8%
Security vulnerabilities	LLM + limited tooling	~0.10	12.2%
Requirements misunderstanding	Cannot be caught by verification alone	~0.05	13.5%

Honest assessment: The 25x improvement claim holds for the top two categories — errors detectable by deterministic tooling. For subtle architectural and security issues (the errors that matter most in production), the improvement is closer to **2-3x**. For requirements misunderstandings, the architecture provides almost no benefit, as the verifier checks against the subtask description, not the user's true intent.

This stratification is critical for setting realistic expectations. Verified Succession is highly effective at catching *mechanical* errors and moderately effective at catching *reasoning* errors. It does not substitute for human architectural review or requirements validation.

5.5.3 Dead-End Elimination

Unique to Verified Succession and quantified empirically: 20-40% of context in complex tasks is consumed by exploration that leads nowhere [17]. All existing context management techniques (RAG, summarization, sliding windows) carry residue of dead ends. Verified Succession eliminates 100% at each succession boundary by starting the successor fresh.

Practical impact: Successor agents do not inherit their predecessors' biases, fixations, or failed approaches. If agent B.2 spent 30% of its context pursuing a wrong hypothesis, B.2' never encounters it. B.2' may discover a fundamentally different — and potentially superior — approach.

5.5.4 Emergent Right-Sizing

The architecture adapts to problem complexity without upfront configuration:

- **Simple tasks:** 1 agent, no spawning, minimal overhead (~$1.75)
- **Moderate tasks:** 3-9 agents, one level of decomposition (~$7-15)
- **Complex tasks:** 20-50 agents, multi-level recursion (~$40-60)
- **Extreme tasks:** 100+ agents, deep recursion (~$80-175)

Google's predictive model for agent architecture selection achieves $R^2 = 0.513$ [25] — their best model explains only about half the variance in which architecture works best. Verified Succession sidesteps this by not choosing a fixed architecture; it discovers the right topology at runtime based on actual problem complexity.

5.5.5 Complete Audit Trail

Every succession event produces a verification report, every decomposition produces a task manifest, and the state store accumulates a complete history of the work. This provides:

- **Debuggability.** When the final output is wrong, the lineage trail shows exactly which agent, at which generation, introduced the error.
- **Improvability.** Patterns in verifier reports (e.g., "Agent generation 1 consistently misses edge cases in permission checks") can inform prompt engineering and agent specialization.
- **Compliance.** For regulated domains, the audit trail demonstrates due diligence in the automated work process.

5.6 The Applicability Decision Framework

Not every task warrants Verified Succession. The following framework identifies when the architecture is justified:

Condition	Single Agent Fits	Single Context Does Not Fit
Low complexity	Single agent wins ($1-2, fast)	Single agent + summary ($2-4, lossy)
Moderate complexity	Single agent sufficient, VS is overkill ($7-8 wasted)	**VS sweet spot** ($8-60, high quality)
High complexity	Impossible	**VS is the only option** ($40-175)

Use Verified Succession when ALL of the following hold:

1. The task demonstrably exceeds or may exceed a single agent's context window
2. The task is decomposable into parallel or tree-structured subtasks
3. The task produces verifiable artifacts (code, structured data, test-passing implementations)
4. Correctness matters more than speed
5. The cost of the agent swarm ($7-175) is justified by the value of the task (measured against human effort or the cost of failure)

Do NOT use Verified Succession when ANY of the following hold:

1. The task fits within a single agent's context (< 50% utilization)
2. The task is inherently sequential and non-decomposable
3. Real-time latency is the primary constraint
4. The task does not produce inspectable artifacts (open-ended brainstorming, creative writing)

5. Budget constraints prohibit the 4-5x minimum cost premium

5.7 The Temporal Caveat: Context Windows Are Growing

An honest assessment must acknowledge the elephant in the room: context windows are expanding rapidly. Claude supports 200K tokens. Gemini supports 1M+. Future models may support 10M or more.

If context windows grow faster than task complexity, the core motivation for Verified Succession weakens. A 10M-token context window with maintained attention quality might handle any reasonable software engineering task in a single session, rendering the architecture unnecessary overhead.

If task complexity grows faster than context windows — which historically it does, as users consistently ask for more ambitious automation — Verified Succession remains relevant regardless of window size. The architecture is window-size agnostic; it activates only when the threshold is crossed, regardless of whether that threshold is at 120K tokens or 6M tokens.

Our assessment: The architecture is most valuable in the current 2025-2027 window, where context limits are large enough to be useful but small enough to be regularly exceeded by complex tasks. It may evolve from a "core architecture" to an "edge-case safety net" as models improve. This is acceptable — good engineering solves today's problems while acknowledging tomorrow's landscape.

5.8 Proposed Evaluation Methodology

To validate the theoretical properties established in this paper, we propose a benchmark suite measuring eight metrics across established software engineering benchmarks (SWE-bench Verified [21], HumanEval [22]):

Metric	Definition	Target vs. Baseline
Task completion rate	% of benchmark tasks fully completed	> single agent by ≥15%
Cost efficiency ratio	completion_rate / total_cost	Justify the 4-5x cost premium
Output error rate	Type errors + test failures + logic bugs in final output	< single agent
Context utilization efficiency	% of consumed context used for productive reasoning	> 80% (vs. ~60-70% single agent)
Succession accuracy	% of verifier reports correctly identifying remaining work	> 85%
Decomposition quality	% of decompositions with no gaps or overlaps in coverage	> 90%
Latency overhead	Wall-clock time relative to single agent baseline	< 3x for typical-complexity tasks
Depth efficiency	Marginal completion rate gain per additional recursion level	Diminishing returns threshold

The CLEAR evaluation framework (Cost, Latency, Efficiency, Assurance, Reliability) [28] provides the appropriate multi-dimensional scoring methodology, as it captures the full cost-quality tradeoff rather than optimizing for accuracy alone. Traditional accuracy-focused evaluation misses cost variations of up to 50x for similar precision levels [28], making cost-normalized metrics essential for assessing production viability.

---

6. Specification-Driven Routing Pipeline

6.1 The Routing Problem

The analysis in Section 5 establishes clear applicability boundaries for Verified Succession, but leaves a critical question unanswered: given a raw problem statement — *"We need this feature"* — how does the system determine which execution architecture to employ? Choosing too simple an approach wastes the initial execution cost when the task fails or degrades. Choosing too complex an approach wastes the overhead premium on a task that did not require it.

Google's predictive model for agent architecture selection achieves $R^2 = 0.513$ [25], meaning even their best automated classifier explains only half the variance in optimal architecture. This suggests that **static heuristics are insufficient** for routing decisions. The problem requires structured analysis of the specific task in context.

More fundamentally, Section 5.5.2 identifies requirements misunderstanding as the weakest error category in the entire architecture ($p_v \approx 0.05$). No amount of post-hoc verification can compensate for building the wrong thing. The routing decision must therefore serve a dual purpose: it must assess complexity *and* harden requirements before any implementation tokens are spent.

6.2 Pipeline Architecture

We introduce a **Specification-Driven Routing Pipeline** that sits upstream of the execution phase. The pipeline converts ambiguous problem statements into validated specification sheets through a structured sequence of planning, human interview, concurrent assessment, and synthesis.

```
SPECIFICATION-DRIVEN ROUTING PIPELINE
═══════════════════════════════════════

Stage 1: PLANNING AGENT
  Input:  Raw problem statement
  Action: Explores codebase, maps dependencies, identifies unknowns
  Output: Draft approach + list of open questions

Stage 2: INTERVIEW PHASE (Human-in-the-Loop)
  Input:  Draft approach + open questions
  Action: Planning agent asks the human stakeholder targeted questions
          to resolve ambiguities, confirm scope, and validate assumptions
  Output: Resolved requirements + design decisions

Stage 3: SPEC SHEET GENERATION
  Input:  Resolved requirements
  Action: Planning agent produces a structured specification artifact
  Output: Spec sheet with acceptance criteria, scope boundaries,
          and technical constraints

Stage 4: CONCURRENT ASSESSMENT (Conditional)
  Input:  Spec sheet
  Action: 3 independent ranking agents evaluate the spec:
          - Complexity assessment (files, systems, dependencies)
          - Gap analysis (missing requirements, ambiguities)
          - Risk identification (untested areas, external dependencies)
  Output: 3 independent findings reports

Stage 5: SYNTHESIS AND ROUTING
  Input:  Spec sheet + 3 findings reports
  Action: Synthesis agent merges assessments, fills identified gaps,
          produces consensus complexity score, and selects execution
          architecture
  Output: Refined spec + routing decision + implementation brief
```

6.3 Stage Descriptions

6.3.1 Stage 1: The Planning Agent

The planning agent performs the "scout" function: it reads relevant source files, maps module dependencies, identifies the subsystems a task will touch, and drafts an initial approach. Critically, it also produces a **list of open questions** — ambiguities in the problem statement that require human input to resolve.

This stage consumes approximately 50K input tokens and 15K output tokens (~$0.63). The planning agent should be instructed to optimize for question quality over plan completeness: a thorough list of unknowns is more valuable than a detailed plan built on assumptions.

6.3.2 Stage 2: The Interview Phase

The planning agent presents its open questions to the human stakeholder in a structured interview. This is the single highest-value step in the entire system.

Example interview for "We need RBAC auth":

```
Planning Agent → Human:
  1. How many roles do you need? (I see admin referenced in the
     existing code, but no others defined)
  2. JWT or session-based? (Current auth uses sessions, but the
     API layer expects stateless tokens)
  3. Should role changes take effect immediately or on next login?
  4. Is audit logging in scope, or separate task?
  5. Do you need row-level security, or just endpoint-level RBAC?
```

Each question surfaces an ambiguity that, if left unresolved, would propagate as a requirements misunderstanding through the entire execution phase. The interview elevates the requirements error detection probability from $p_v \approx 0.05$ (post-hoc verification) to $\mathbf{p_v \approx 0.90+}$ (direct human validation), because the human stakeholder is the authoritative source of intent.

This stage costs approximately 30K input tokens and 10K output tokens (~$0.40), plus human time. The human time investment is small (typically 5-10 minutes of answering targeted questions) but produces outsized returns by preventing wrong-thing-built failures that would cost $7-175 to re-execute.

6.3.3 Stage 3: Spec Sheet Generation

The planning agent synthesizes the interview responses into a **structured specification artifact** — a formal document with:

- **Scope definition.** Exactly what is included and excluded.
- **Acceptance criteria.** Testable conditions that define "done."
- **Technical constraints.** Frameworks, patterns, and conventions to follow.
- **Dependency map.** Files, modules, and systems that will be touched.
- **Interface contracts.** How the new code will integrate with existing code.

The spec sheet is the central artifact of the routing pipeline. It serves as the single source of truth for all downstream agents, whether the task is executed by a single agent or a Verified Succession swarm. Every agent in the execution phase receives this spec — not the raw problem statement.

This stage costs approximately 20K input tokens and 8K output tokens (~$0.30).

6.3.4 Stage 4: Concurrent Assessment

Three independent ranking agents evaluate the spec sheet in parallel. Each agent performs the same analysis but arrives at independent conclusions — a concurrent pattern [9] applied to specification analysis rather than implementation.

Each ranking agent produces a structured findings report:

```
RANKING AGENT REPORT SCHEMA
──────────────────────────

{
  complexity_score: 1-10,
  complexity_rationale: "...",

  files_estimated: N,
  subsystems_touched: ["auth", "db", "api"],
  estimated_context_requirement: "percentage of C_max",

  gaps_found: [
    { description: "...", severity: "critical|moderate|minor" }
  ],

  risks_found: [
    { description: "...", mitigation: "..." }
  ],

  decomposability: "high|medium|low",
  recommended_architecture: "single|single+verifier|vs-shallow|vs-deep",
  confidence: 0.0-1.0
}
```

When to skip this stage: Not every spec requires three agents to evaluate. The planning agent self-assesses its confidence in the spec's completeness:

- **Confidence > 90%:** Skip Stage 4. Route directly based on the planning agent's complexity estimate. Appropriate for simple tasks ("add a button", "fix this typo").
- **Confidence 60-90%:** Execute Stage 4. The spec has sufficient structure for ranking agents to evaluate but contains enough complexity to benefit from multiple perspectives.
- **Confidence < 60%:** Return to Stage 2. The planning agent itself is uncertain — the spec needs more human input before agents should evaluate it.

This stage costs 3 × (40K input + 10K output) ≈ $1.35 when invoked.

6.3.5 Stage 5: Synthesis and Routing

The synthesis agent receives the spec sheet and all three findings reports. It performs four functions:

1. **Consensus assessment.** If all three agents agree on complexity (within ±2 points), high confidence in the estimate. If scores diverge widely (e.g., 4, 7, 9), this signals that the spec is ambiguous — different readers interpret it differently — and the pipeline should loop back to Stage 2.
2. **Gap resolution.** Gaps identified by 2+ agents are confirmed and incorporated into a refined spec. Gaps identified by only 1 agent are flagged but not necessarily incorporated.
3. **Risk aggregation.** Risks flagged by multiple agents receive higher priority. The synthesis agent assesses whether any risks should block execution or merely inform the implementation approach.

4. **Routing decision.** Based on the consensus complexity score:

Consensus Score	Context Estimate	Routing Decision
1-3	< 40% of C_max	Single agent with spec sheet
4-5	40-70% of C_max	Single agent + post-execution verifier pass
6-8	70-200% of C_max	Verified Succession — shallow (1 level)
9-10	> 200% of C_max	Verified Succession — deep (multi-level)

The synthesis agent produces:
- **Refined spec sheet** with confirmed gaps filled
- **Routing decision** with justification
- **Implementation brief** tailored to the selected architecture (e.g., suggested decomposition boundaries for VS tasks)

This stage costs approximately 60K input tokens and 15K output tokens (~$0.68).

6.4 Cost of the Routing Pipeline

Stage	Token Consumption	Cost	When Invoked
Planning Agent	50K in, 15K out	~$0.63	Always
Interview Phase	30K in, 10K out	~$0.40	Always
Spec Sheet Generation	20K in, 8K out	~$0.30	Always
3 Ranking Agents	120K in, 30K out	~$1.35	Confidence 60-90%
Synthesis Agent	60K in, 15K out	~$0.68	When Stage 4 runs
Total (with ranking)	**280K in, 78K out**	**~$3.36**	
Total (without ranking)	**100K in, 33K out**	**~$1.33**	

Amortization analysis: The pipeline adds $1.33-3.36 to every task. For a simple $1.75 task, this roughly doubles the cost (acceptable only if the interview phase catches a misunderstanding that would otherwise require re-execution). For a complex $40-175 VS task, it represents 2-8% overhead — trivially justified by the routing accuracy and requirements hardening it provides.

Breakeven: If the pipeline prevents even one wrong-thing-built failure per ~3 tasks — which would require a full re-execution at $7-175 — it pays for itself.

6.5 How the Pipeline Addresses Requirements Misunderstanding

The Verified Succession architecture's error attenuation (Theorem 4) is powerful for mechanically detectable errors but nearly ineffective for requirements misunderstandings ($p_v \approx 0.05$, Section 5.5.2). The Specification-Driven Routing Pipeline directly targets this weakness:

Error Category	Without Pipeline (p_v)	With Pipeline (p_v)	Improvement
Type errors	~0.99	~0.99	No change (already maximal)
Logic errors (test-catchable)	~0.85	~0.85	No change
Subtle logic errors	~0.40	~0.45	Marginal (spec clarity helps)
Architectural mistakes	~0.15	~0.50	3x (ranking agents catch structural issues)
Security vulnerabilities	~0.10	~0.15	Marginal
Requirements misunderstanding	**~0.05**	**~0.90**	**18x** (human validates intent)

The 18x improvement in requirements error detection is the pipeline's primary value proposition. It converts the architecture's single weakest error category from "nearly undetectable" to "reliably caught" by positioning human judgment at the point where it is most effective: before implementation begins.

6.6 Divergence Handling

The pipeline includes two explicit feedback loops for handling uncertainty:

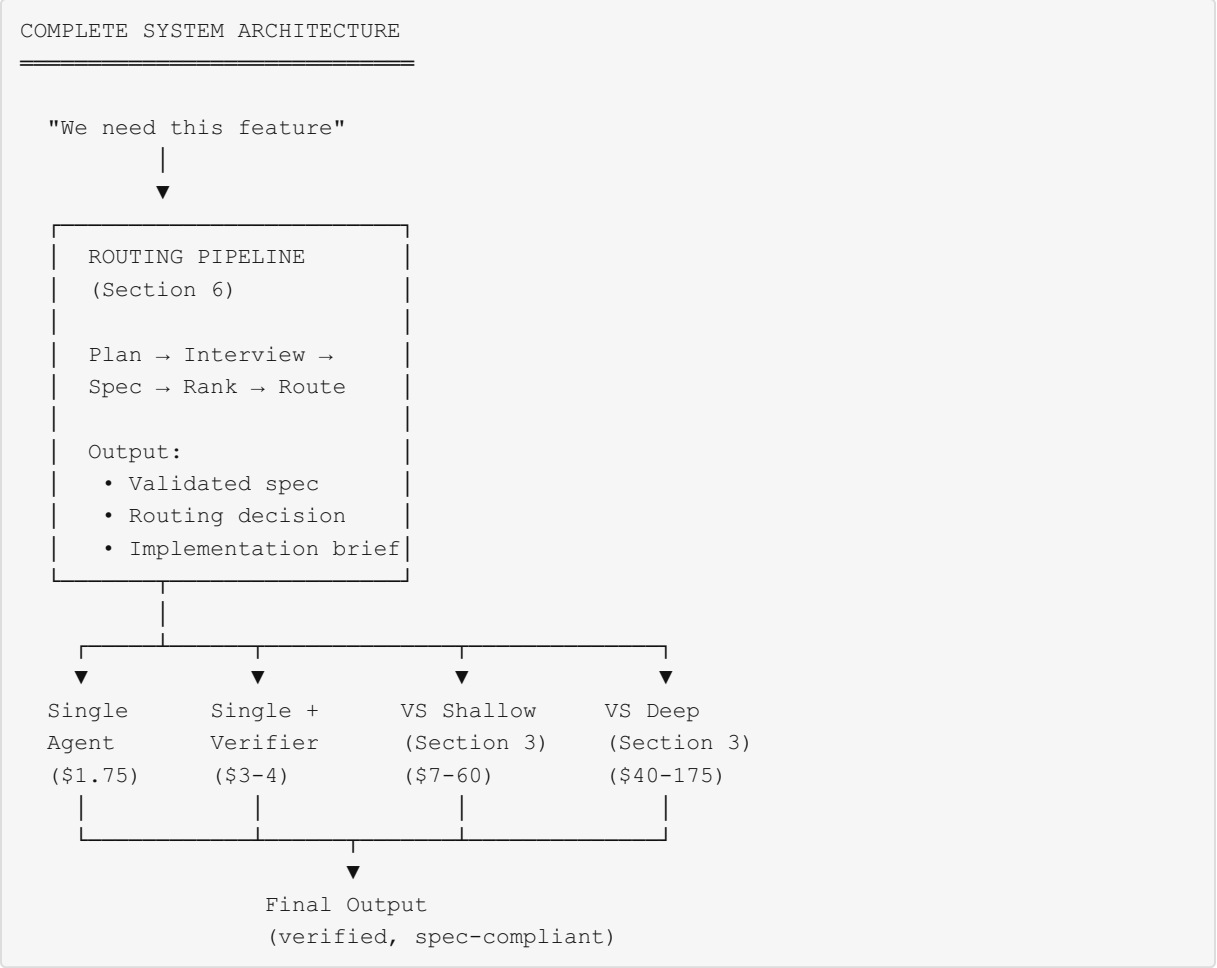
Loop 1: Interview Insufficiency. If the planning agent's confidence remains below 60% after Stage 2, the interview did not resolve enough ambiguity. The pipeline re-enters Stage 2 with more targeted questions informed by what was learned. Maximum 2 re-entries before escalating to "this task requires a human planning session, not an automated pipeline."

Loop 2: Ranking Divergence. If the three ranking agents' complexity scores diverge by more than 4 points (e.g., scores of 3, 5, and 9), the spec is ambiguous enough that three competent readers interpret it fundamentally differently. The synthesis agent flags this and either:
- Identifies the specific section of the spec causing divergence and loops back to Stage 2 for clarification
- Recommends splitting the task into sub-tasks that can be independently assessed

These feedback loops ensure the pipeline does not route confidently on insufficient information. The cost of an additional interview round (~$0.40) is negligible compared to the cost of misrouted execution.

6.7 Integration with Verified Succession

The routing pipeline and execution architecture form a complete system:



Each execution path receives the **same validated spec sheet** — ensuring that regardless of which architecture is selected, the agents are building the right thing. The spec sheet replaces the raw problem statement as the authoritative input to the execution phase.

For Verified Succession specifically, the spec sheet improves the architecture's operation at every level:

- **Decomposition quality improves** because the spec's dependency map and interface contracts inform where to split work, reducing the risk of bad decompositions (Section 9.1).
- **Verifier accuracy improves** because acceptance criteria from the spec provide objective pass/fail conditions, augmenting the verifier's LLM judgment with concrete checkpoints.
- **Aggregation quality improves** because the sub-orchestrator can validate completeness against the spec's scope definition rather than inferring it from the original problem statement.

---

7. Application to Software Engineering

7.1 Why Software Engineering Demands This Architecture

Software engineering tasks exhibit three properties that make them uniquely suited to Verified Succession:

1. **Unpredictable complexity.** A task described as "fix the authentication bug" may require reading 2 files or 50. The required agent topology cannot be determined from the task description alone.
2. **Artifact-oriented output.** Software engineering produces discrete, inspectable artifacts (code files, test results, configuration changes). These are naturally suited to the state store model — the verifier can run tests, check types, and diff outputs to verify correctness objectively.
3. **Compositional structure.** Software tasks decompose naturally: a feature can be split into backend API, frontend UI, database migration, and test suite. Each subtask has clear boundaries and interfaces.

7.2 Concrete Workflow: Feature Implementation

Consider implementing a user authentication system with role-based access control. The Verified Succession architecture would proceed as follows:

```
ORCHESTRATOR receives: "Implement RBAC authentication system"
│
├── Agent_Auth (concurrent): "Design and implement auth middleware"
│   │   [hits 0 at 60% — task requires JWT + session + RBAC + audit]
│   │
│   └── Sub-Orchestrator_Auth
│       ├── Agent_JWT: "Implement JWT validation and refresh"
│       │   [completes within context budget → returns artifacts]
│       │
│       ├── Agent_RBAC: "Implement role-based access control"
│       │   [hits 0 — 4 roles × permissions matrix is large]
│       │   │
│       │   ├── writes artifacts: admin + editor roles done
│       │   ├── terminates (generation 1)
│       │   │
│       │   ├── Agent_RBAC' spawns (generation 2)
│       │   │   ├── Verifier reads RBAC artifacts
│       │   │   ├── Verifier report:
│       │   │   │   "✓ admin role complete, ✓ editor role complete,
│       │   │   │    ✗ viewer role missing, ✗ auditor role missing,
│       │   │   │    ⚠ permission enum doesn't include AUDIT_READ"
│       │   │   ├── Verifier terminates
│       │   │   │
│       │   │   └── Agent_RBAC' continues: implements viewer + auditor
│       │   │       [completes within context budget → returns artifacts]
│       │   │
│       │   └── [RBAC subtask complete]
│       │
│       └── Agent_Audit: "Implement authentication audit logging"
│           [completes within context budget → returns artifacts]
│
├── Agent_Tests (concurrent): "Write comprehensive test suite"
│   [proceeds independently, may also spawn sub-swarm]
│
├── Agent_Migration (concurrent): "Create database migration for auth tables"
│   [completes within context budget → returns artifacts]
│
└── ORCHESTRATOR aggregates: middleware + tests + migration = complete feature
```

7.3 Verification in Software Engineering Context

The verification sub-agent has unique advantages in software engineering because verification can be **objective and automated**:

Verification Method	What It Catches
Type checking (`tsc` , `cargo check`)	Structural errors, missing implementations
Test execution	Behavioral regressions, logic errors
Linting	Style violations, potential bugs
Diff analysis	Unintended changes, scope drift
Dependency analysis	Missing imports, circular dependencies
Schema validation	API contract violations

The verifier is not limited to LLM judgment. It can invoke deterministic tools to produce high-confidence assessments. This addresses a key weakness of LLM-only verification: the verifier's `p_v` (error detection probability) approaches 1.0 for categories of errors that tooling can catch, dramatically improving the error attenuation properties derived in Section 4.4. See Section 5.5.2 for a stratified analysis of `p_v` by error category.

7.4 State Store Design for Software Artifacts

In the software engineering domain, the state store maps naturally to the filesystem and version control:

```
state-store/
├── manifests/
│   ├── auth-middleware.json       # Task manifest
│   └── rbac-implementation.json   # Subtask manifest
├── artifacts/
│   ├── agent-rbac-gen1/
│   │   ├── src/middleware/rbac.ts  # Code artifact
│   │   ├── src/types/roles.ts     # Code artifact
│   │   └── self-assessment.json   # Agent's self-report
│   └── agent-rbac-gen2/
│       ├── src/middleware/rbac.ts  # Updated code
│       ├── src/types/roles.ts     # Updated types
│       └── self-assessment.json
├── verifications/
│   ├── rbac-gen1-verification.json  # Verifier report
│   └── test-results-gen1.json       # Automated test output
└── lineage/
    └── rbac.json                    # Succession chain metadata
```

Git branches provide a natural isolation mechanism: each agent (or agent generation) can operate on a dedicated branch, with the sub-orchestrator handling merges. This aligns with worktree-based isolation patterns already established in production agentic development workflows.

---

8. Comparison with Existing Patterns

Property	Sequential	Concurrent	Handoff	Group Chat	Magentic	Verified Succession
Topology	Fixed pipeline	Fixed fan-out	Dynamic chain	Fixed group	Adaptive plan	**Emergent fractal**
Context handling	Accumulates	Independent	Transfers	Shared thread	Accumulates	**Fresh per generation**
Depth	1	1	1	1	1	**Unbounded (configurable)**
Error correction	None	At aggregation	None	Via debate	Via replanning	**At every succession**
Dead-end filtering	None	N/A	None	Partial (debate)	Partial (replan)	**Complete**
Spawning trigger	Predetermined	Predetermined	Agent decision	Predetermined	Manager decision	**Context utilization**
Cost predictability	High	High	Medium	Medium	Low	**Bounded (Theorem 3)**
Cost multiplier vs. single	1x	2-3x	1-2x	2-4x	3-10x	**4-100x**
Latency multiplier vs. single	1-2x	0.5-1x	1-3x	2-5x	3-10x	**2-8x**
Best suited for	Pipelines	Independent analysis	Unknown routing	Consensus	Open-ended	**Long-horizon, complex**

The key differentiator is the combination of **emergent topology** and **verified handoff**. No existing pattern provides both. However, the cost and latency multipliers are the highest of any pattern — this is the explicit tradeoff for the capability unlock.

---

9. Limitations and Open Problems

9.1 Decomposition Quality

The architecture's effectiveness depends critically on the quality of task decomposition at each spawning event. A poor decomposition — overlapping subtasks, missing coverage, or artificially coupled subtasks — degrades the entire sub-swarm. The decomposition occurs when the agent is already at 60% context utilization, meaning it must perform high-quality meta-reasoning with limited remaining capacity.

Mitigation: The adaptive threshold (Proposition 2) helps by triggering decomposition earlier for agents with high tool density. Additionally, the verifier at the aggregation stage can detect gaps between subtask coverage and the original task.

9.2 Latency

Each succession event introduces latency: artifact writing, verifier spawning, artifact reading, report generation, verifier termination, and successor initialization. For time-sensitive applications, this overhead may be prohibitive. See Section 5.3 for detailed latency measurements.

9.3 Verifier Reliability

The architecture assumes verifiers are more reliable than the agents whose work they verify. This holds when verification is easier than generation (a well-established asymmetry in computational complexity) and when deterministic tools (type checkers, test suites) supplement LLM judgment. However, for tasks where verification is as hard as generation (e.g., assessing the correctness of a novel algorithm), verifier reliability may be insufficient. See Section 5.5.2 for stratified reliability analysis by error category.

9.4 State Store Consistency

In deeply recursive architectures with concurrent agents writing to the same state store, consistency guarantees become important. Two agents in the same sub-swarm must not write conflicting artifacts to the same state store paths. The sub-orchestrator must enforce isolation between concurrent sibling agents.

9.5 Cost

While resource consumption is bounded (Theorem 3), the bound can still be large. See Section 5.2 for concrete cost analysis across scenarios. The architecture is most cost-effective for high-value, long-horizon tasks where the alternative (human effort or agent failure) is more expensive.

9.6 Non-Decomposable Tasks

Google's scaling study [25] demonstrates that sequential, non-decomposable tasks experience 39-70% performance degradation under multi-agent orchestration. Verified Succession inherits this limitation — if a task cannot be meaningfully decomposed into independent subtasks, the architecture's overhead exceeds its benefit. The decomposition phase must include a "no-decompose" exit path where the agent determines that the remaining work is atomic and continues without spawning.

---

10. Future Work

10.1 Adaptive Branching Factor

The current architecture uses a fixed branching factor during decomposition. An adaptive approach would analyze subtask complexity estimates and adjust `k` accordingly — simple tasks get fewer sub-agents, complex tasks get more.

10.2 Cross-Swarm Learning

When multiple top-level agents independently discover the same blockers or patterns, this information could be shared across swarms in real time. A shared knowledge bus (distinct from the state store) could propagate insights like "the session store API requires authentication tokens" to all active agents.

10.3 Verification Caching

If agent `α.j''` (generation 3) is spawned for the same subtask, and the work from generation 1 was already verified, the generation 3 verifier need not re-verify generation 1's artifacts — only generation 2's. A verification cache in the state store could avoid redundant auditing.

10.4 Hybrid Model Selection

Not all agents require the most capable (and expensive) model. Verifiers performing structured assessment could use smaller, faster models (e.g., Haiku-tier at $1/$5 per million tokens vs. Opus-tier at $5/$25). Sub-orchestrators performing coordination logic could similarly use lighter models. This hybrid approach could reduce typical-case costs by 30-50% without meaningfully impacting quality.

10.5 Dynamic Turn Limits

Recent work [27] demonstrates that dynamic turn limits based on success probability can reduce agent costs by 24% while maintaining solve rates. Integrating this with the context monitor could provide a second dimension of cost control: not just "when to decompose" but "when to stop trying and escalate."

10.6 Empirical Evaluation

This paper presents the architecture and theoretical analysis. Empirical evaluation on established benchmarks (SWE-bench Verified [21], BrowseComp [4], HumanEval [22]) is needed to validate the theoretical properties and establish practical performance characteristics. The evaluation methodology proposed in Section 5.8 provides the framework for this work.

---

11. Conclusion

We have presented **Verified Succession**, a recursive context-aware multi-agent architecture that treats context window exhaustion as a structural spawning signal rather than a failure condition, together with the **Specification-Driven Routing Pipeline** that determines the appropriate execution architecture for each task.

The verify-then-continue protocol is the central execution contribution. By having successor agents independently audit their predecessors' work through short-lived verification sub-agents, the architecture achieves three properties simultaneously: (1) near-complete context headroom for successor agents (~90-95%), (2) exponential error attenuation at each succession boundary, and (3) complete elimination of dead-end tokens. No existing orchestration pattern provides all three.

The Specification-Driven Routing Pipeline is the central planning contribution. By positioning human-in-the-loop requirements validation upstream of execution, the pipeline addresses the architecture's single weakest error category — requirements misunderstanding — elevating its detection probability from $p_v \approx 0.05$ to $p_v \approx 0.90+$ (an 18x improvement). The concurrent multi-agent assessment of the resulting spec sheet produces a high-confidence routing decision while simultaneously hardening the specification that all downstream agents will work from.

However, these gains come at a quantified cost. The execution architecture introduces a 4-5x cost multiplier for typical tasks and 2-8x latency overhead compared to single-agent approaches. The routing pipeline adds $1.33-3.36 per task. Error attenuation is highly effective for mechanically detectable errors (type errors, test failures) but provides diminishing returns for subtle architectural and security issues. The combined system is not universally superior — it is a specialized tool for a specific class of problems: complex, decomposable, artifact-producing tasks that exceed single-agent capacity and justify the routing and execution overhead.

The honest value proposition is this: the combined system trades upfront planning cost (~$3) and execution premium (4-5x) for three guarantees: (1) the right thing is built (validated spec), (2) it is built correctly (verified succession), and (3) tasks that would otherwise fail due to context exhaustion are completed (emergent topology). Whether that trade is justified depends entirely on what is on the other side of the task. For a $100/hour engineer spending 4 hours on a task, paying $10-65 for an agent system that completes it correctly in 15 minutes is an obvious trade. For a quick configuration change, it is unnecessary overhead.

As LLM agents are increasingly deployed for complex, long-horizon software engineering tasks, architectures that gracefully scale with problem complexity — rather than failing at fixed context boundaries — will become essential. The combination of Specification-Driven Routing and Verified Succession provides a principled, honestly quantified framework for building such systems: one that decides *what to build* before deciding *how to build it*.

References

[1] Chen, M., et al. “Evaluating Large Language Models Trained on Code.” *arXiv preprint arXiv:2107.03374* (2021).

[2] Jimenez, C.E., et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” *arXiv preprint arXiv:2310.06770* (2023).

[3] Anthropic. “Claude Model Card and Evaluations.” *Anthropic Technical Report* (2025).

[4] Anthropic. “BrowseComp: A Benchmark for Browsing Comprehension.” *Anthropic Research* (2025).

[5] Liu, N.F., et al. “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the Association for Computational Linguistics* (2024).

[6] Park, J.S., et al. “Generative Agents: Interactive Simulacra of Human Behavior.” *UIST* (2023).

[7] Wu, Q., et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” *arXiv preprint arXiv:2308.08155* (2023).

[8] Hong, S., et al. “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework.” *ICLR* (2024).

[9] Kittel, C. and Siemens, C. “AI Agent Orchestration Patterns.” *Microsoft Azure Architecture Center* (2026).

[10] Kanerika. “AI Agent Orchestration in 2026: Coordination, Scale and Strategy.” *Kanerika Research* (2026).

[11] Google. “Developer’s Guide to Multi-Agent Patterns in ADK.” *Google Developers Blog* (2025).

[12] Salesforce. “Enterprise Agentic Architecture and Design Patterns.” *Salesforce Architects* (2025).

[13] Li, W., et al. “AgentSpawn: Adaptive Multi-Agent Collaboration Through Dynamic Spawning for Long-Horizon Code Generation.” *arXiv preprint arXiv:2602.07072* (2026).

[14] Kimi. “Kimi Agent Swarm: 100 Sub-Agents at Scale.” *Kimi Blog* (2025).

[15] Sifry, D. “MetaSwarm: A Self-Improving Multi-Agent Orchestration Framework.” *GitHub* (2025).

[16] Zhang, Y., et al. “Solving Context Window Overflow in AI Agents.” *arXiv preprint arXiv:2511.22729* (2025).

[17] LangChain. “Context Engineering in Agents.” *LangChain Documentation* (2025).

[18] Megagon AI. “VERIMAP: Verification-Aware Planning for Multi-Agent LLM Systems.” *Megagon Research* (2025).

[19] Wang, X., et al. “Multi-Agent Code Verification via Information Theory.” *arXiv preprint arXiv:2511.16708* (2025).

[20] Karapetyan, G. “Tackling the Partial Completion Problem in LLM AI Agents.” *Medium* (2025).

[21] Jimenez, C.E., et al. “SWE-bench Verified: A Stricter Evaluation of Real-World Software Engineering.” *arXiv preprint* (2024).

[22] Chen, M., et al. “HumanEval: Hand-Written Evaluation Set for Code Generation.” *OpenAI* (2021).

[23] Kim, S., et al. “Token Distribution of LLM Multi-Agent Systems.” *OpenReview* (2025).

[24] Liu, X., et al. “How Do Coding Agents Spend Your Money? Analyzing and Predicting Token Consumptions in Agentic Coding Tasks.” *OpenReview* (2025).

[25] Google Research. “Towards a Science of Scaling Agent Systems: When and Why Agent Systems Work.” *Google Research Blog* (2025).

[26] Anthropic. “Claude API Pricing.” *Anthropic Platform Documentation* (2026).

[27] Stevens Institute. “The Hidden Economics of AI Agents: Managing Token Costs and Latency Trade-offs.” *Stevens Online* (2025).

[28] Galileo AI. “Benchmarking Multi-Agent AI: Insights and Practical Use.” *Galileo Blog* (2025).

Correspondence: Dylan Schwindt. This work is independent and not affiliated with any institution. The Verified Succession architecture was developed through collaborative discourse on agentic system design, February 2026.